

JAPANCAR PARTS — Plano de Produção

Este documento descreve a infraestrutura técnica necessária para colocar o marketplace JAPANCAR PARTS em produção. Aqui você encontra uma visão completa dos serviços, custos e configurações necessárias para operar o aplicativo de forma profissional e escalável.

1. Visão Geral — O que é infraestrutura de produção?

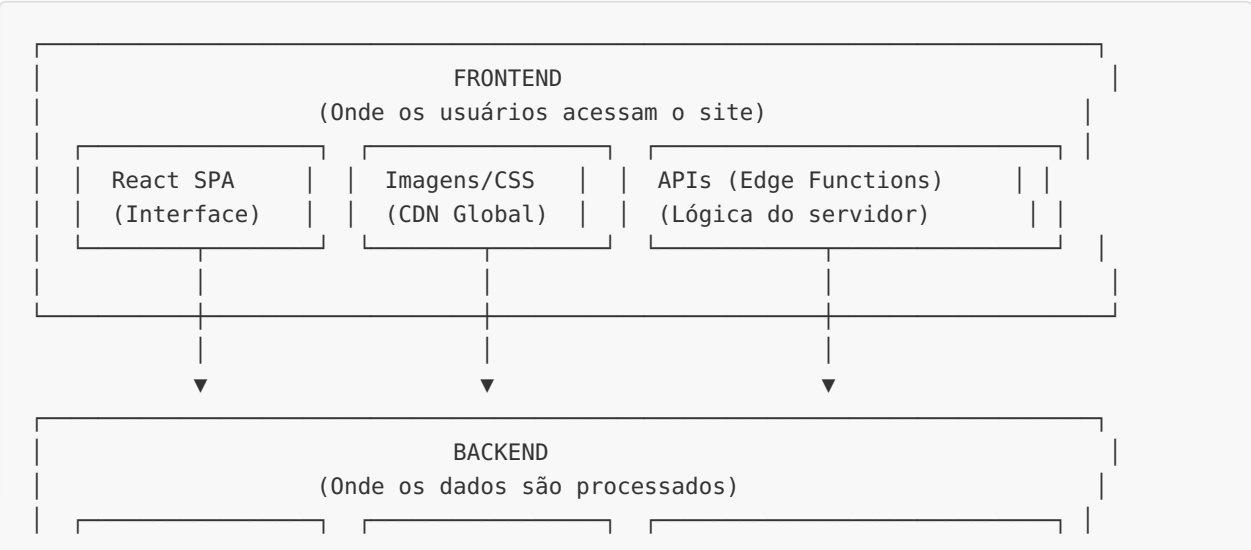
1.1 Por que precisamos de infraestrutura de produção?

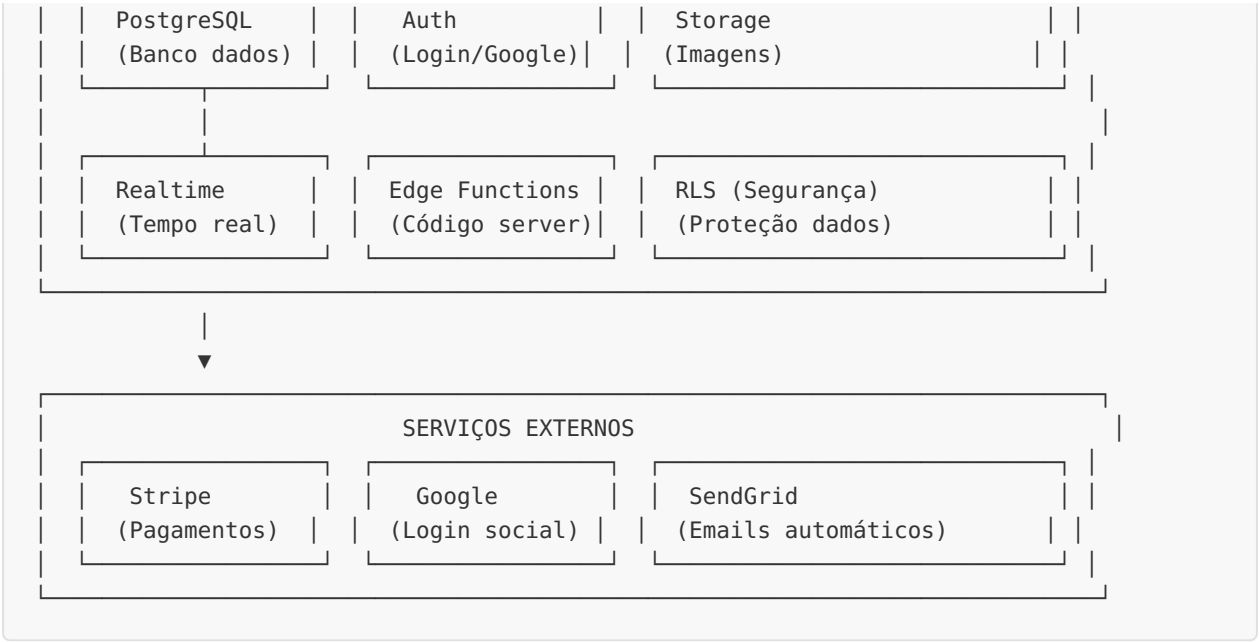
Quando desenvolvemos um aplicativo localmente (no nosso computador), ele funciona, mas não está disponível para outros usuários acessarem pela internet. Para que clientes reais possam usar o sistema, precisamos de:

- 1. Um **servidor** para hospedar o código (Frontend)
- 2. Um **banco de dados** para armazenar informações (Backend)
- 3. **Serviços de segurança** para proteger dados e transações
- 4. **Ferramentas de pagamento** para processar compras

Tudo isso juntos forma a **infraestrutura de produção**.

1.2 Arquitetura Simplificada





2. Detalhamento da Infraestrutura

2.1 Frontend — Vercel (Hospedagem do Site)

O que é: O Vercel é uma plataforma de hospedagem para aplicações web. Ele serve o site para usuários de todo o mundo através de uma rede global de servidores (CDN), garantindo velocidade e disponibilidade.

Por que este escolha: - Integração nativa com React/Vite (nosso stack) - SSL/gratuito automático - Deploy automático via GitHub - CDN global para imagens e arquivos estáticos

Configurações técnicas:

Recurso	Valor	Explicação
Plataforma	Vercel	Hospedagem profissional
Tipo	SPA	Single Page Application (sem recarregar página)
Framework	React 19 + TypeScript + Vite	Stack atual do projeto
CDN	Global edge network	Servidores em todo o mundo
SSL	Automático (Let's Encrypt)	Conexão segura
Domínio	japancarparts.jp	Domínio profissional

Configuração de Deploy (`vercel.json`):

```
{
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "framework": "vite",
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

Variáveis de Ambiente (configurar no painel Vercel):

```
VITE_SUPABASE_URL=https://<project>.supabase.co
VITE_SUPABASE_ANON_KEY=<sua-chave-pública>
VITE_STRIPE_PUBLIC_KEY=pk_live_...
VITE_GOOGLE_CLIENT_ID=<seu-client-id>
VITE_APP_URL=https://japancarparts.jp
```

Metas de Performance (Core Web Vitals):

Métrica	O que significa	Meta
LCP	Tempo para carregar maior elemento visível	< 2.5s
FCP	Tempo para primeiro conteúdo visível	< 1.8s
TTFB	Tempo até primeiro byte do servidor	< 600ms
CLS	Estabilidade visual (sem movimentos)	< 0.1
FID	Tempo de resposta a cliques	< 100ms

2.2 Backend — Supabase (Banco de Dados e Serviços)

O que é: O Supabase é uma plataforma "Backend-as-a-Service" que oferece: - Banco de dados PostgreSQL (o mesmo usado por empresas como Netflix, Spotify) - Sistema de autenticação - Storage para arquivos (imagens) - Realtime (atualizações em tempo real) - Edge Functions (código serverless)

Por que esta escolha: - Região Tokyo disponível (latência baixa para usuários japoneses) - Plano Pro tem recursos suficientes para começar - Já estamos usando no projeto (mesmo stack) - Segurança robusta com RLS nativa

Database (PostgreSQL)

Recurso	Plano Pro	Explicação
Região	Tokyo (ap-northeast-1)	Servidor no Japão = 速度快

Recurso	Plano Pro	Explicação
Storage	8 GB	Espaço para dados
Bandwidth	50 GB/mês	Dados transferidos por mês
Backup	Automático (7 dias)	Restore disponível por 7 dias
PITR	Enterprise (opcional)	Restore para qualquer momento

Tabelas do Banco:

public/	
├─ profiles	→ Dados dos usuários (nome, email, rating, etc)
├─ brands	→ Marcas de carros (Nissan, Toyota, Honda...)
├─ car_models	→ Modelos por marca (GT-R, Supra, NSX...)
├─ categories	→ Categorias de peças (Body Kits, Wheels, etc)
├─ parts	→ Anúncios de peças à venda
├─ bids	→ Lances em leilões
├─ transactions	→ Vendas realizadas (comissões aqui)
├─ messages	→ Mensagens entre usuários
├─ favorites	→ Itens favoritados
└─ reviews	→ Avaliações após vendas

Índices (otimizações para speed):

-- Busca rápida por filtros comuns
CREATE INDEX idx_parts_brand ON parts(brand_id);
CREATE INDEX idx_parts_category ON parts(category_id);
CREATE INDEX idx_parts_status ON parts(status);
CREATE INDEX idx_parts_price ON parts(price);
-- Leilões ativos
CREATE INDEX idx_parts_auction_end ON parts(auction_end) WHERE status = 'active';
-- Busca em texto (para procurar por título/descrição)
CREATE INDEX idx_parts_search ON parts USING gin(to_tsvector('japanese', title ' ' description));

Autenticação (Login)

Configuração	Valor	Explicação
Provedores	Email/Password + Google OAuth	Login tradicional + Google
MFA	Opcional	Autenticação em 2 fatores (recomendado para lojistas)
Session	7 dias	Tempo que usuário permanece logado

Configuração	Valor	Explicação
Password	Mín 8 caracteres	Política de senha

Storage (Imagens)

O Supabase Storage guarda imagens de peças, avatares e logos.

Bucket	Uso	Limite	Cache
parts-images	Fotos das peças	10 MB/imagem	1 dia
avatars	Fotos de perfil	2 MB/imagem	30 dias
brands-logos	Logos das marcas	1 MB/imagem	1 ano

Otimização: redimensionamento automático via Edge Functions + WebP

Realtime (Atualizações em Tempo Real)

Permite que usuários vejam novos lances e mensagens instantaneamente.

Configuração	Valor
Conexões Simultâneas	200 (Plano Pro)
Channels	parts, bids, messages

Edge Functions (Código Serverless)

São funções que rodam no servidor quando solicitadas. Precisamos de:

Função	Função
parts	Criar, listar, editar peças
users	Gerenciar perfis
transactions	Processar vendas e comissões
auctions	Gerenciar leilões
categories	Listar categorias
brands	Listar marcas
process-payment	Receber confirmações do Stripe

Função	Função
calculate-fees	Calcular taxas de transação

2.3 Pagamentos — Stripe

O que é: O Stripe é a plataforma de pagamentos mais usada por startups. Permite aceitar cartões de crédito, debit, e métodos japoneses (Konbini, PayPay).

Fluxo de pagamento (sistema de custódia):

1. Comprador escolhe peça → Clica em "Comprar"
2. → Redirecionado para página do Stripe
3. → Paga com cartão/outro método
4. → Stripe RETÉM o valor (não paga seller ainda)
5. → Vendedor envia a peça
6. → Comprador recebe e APROVA no sistema
7. → Stripe TRANSFERE: valor - comissão(10%) - taxa Stripe
8. → Vendedor recebe saldo líquido na conta

Configurações:

Configuração	Valor
Modo	Connected Accounts (cada vendedor tem conta vinculada)
Checkout	Stripe Checkout (página redirecionada)
Taxa Stripe	2.9% + ¥30 por transação

Exemplo de breakdown (venda de ¥10.000):

Item	Valor (JPY)	Valor (USD*)
Preço da peça	¥10,000	~\$67
Comissão plataforma (10%)	-¥1,000	-\$6.70
Taxa Stripe (2.9% + ¥30)	-¥320	-\$2.15
Taxa plataforma (operacional)	-¥30	-\$0.20
Total taxes	-¥1,350	-\$9.05
Vendedor recebe	¥8,650	~\$58

Conversão aproximada: 1 USD ≈ 149 JPY

2.4 Segurança

Protegemos o sistema em múltiplas camadas:

Camada	O que faz	Implementação
TLS 1.3	Criptografa conexão (https)	Obrigatório em todas requisições
HSTS	Força https permanentemente	max-age=31536000
RLS	Controla acesso linhas do banco	Policies em cada tabela
Rate Limiting	Limita requisições por IP	100 req/min
Input Validation	Valida dados []	Zod em todas Edge Functions
DDOS Protection	Protege ataques	Vercel + Supabase built-in

Exemplo de política RLS (proteção de dados):

```
-- Qualquer pessoa pode VER peças
CREATE POLICY "parts_read_public" ON parts FOR SELECT USING (true);

-- Apenas o dono pode CRIAR uma peça
CREATE POLICY "parts_insert_owner" ON parts
  FOR INSERT WITH CHECK (auth.uid() = seller_id);

-- Apenas o dono pode EDITAR sua peça
CREATE POLICY "parts_update_owner" ON parts
  FOR UPDATE USING (auth.uid() = seller_id);
```

2.5 Monitoramento

Ferramentas para saber se tudo está funcionando:

Ferramenta	O que faz	Custo
Vercel Analytics	Performance, Core Web Vitals	Grátis
Supabase Logs	Erros de banco e queries lentas	Grátis
Sentry	Erros no código frontend/backend	Grátis (20k/mês)
Uptime Robot	Verificar se site está online	Grátis

2.6 Backup e Recuperação

Recurso	Frequência	Retenção	Para que serve
Database backup	Automático contínuo	7 dias	Restaurar banco se algo corrupto
PITR	Sob demanda (Enterprise)	30 dias	Restaurar para momento específico
Export SQL	Semanal	Manual	Backup offline
Imagens	N/A (já redundante)	-	Storage já tem redundância

2.7 CI/CD Pipeline (Deploy Automático)

Pipeline = conjunto de passos automáticos que acontecem quando você faz push no código.

```
name: Deploy Production

on:
  push:
    branches: [main] # Quando enviar código para main

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'
      - run: npm ci # Instala dependências
      - run: npm run lint # Verifica código
      - run: npm run typecheck # Verifica tipos
      - run: npm run test # Roda testes

  build:
    needs: test # Só executa se test passar
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      - run: npm ci && npm run build # Compila projeto
      - uses: amondnet/vercel-action@v25
        with:
          vercel-token: ${ secrets.VERCEL_TOKEN }
          vercel-args: '--prod' # Deploy para produção
```

3. Custos Mensais — Produção

3.1 Custos Detalhados (Mês 1)

Serviço	Plano	Custo (JPY)	Custo (USD)	Para que serve
Vercel	Free	¥0	\$0	Hospedagem do site + CDN
Supabase	Free	¥0	\$0	Banco + Auth + Storage + Realtime
Stripe	Standard	Fee passed*	-	Taxa cobrada do vendedor
Domínio (.jp)	Registro	¥250/mês	\$2	Domínio profissional
SendGrid	Free Tier	¥0	\$0	Emails transacionais (até 100/mês)
Mão de Obra	Engenheiro	R\$200/sem	~\$40/mês	Desenvolvimento
TOTAL		¥250/mês	~\$42/mês	

*Taxa do Stripe é passada para o vendedor (2.9% + ¥30 por transação)

Nota importante: No estágio atual de desenvolvimento, usamos as camadas gratuitas do Vercel e Supabase, tornando os custos de infraestrutura praticamente zero. O único custo real é a mão de obra de desenvolvimento.

3.2 Evolução de Custos (Projeção)

Mês	Fase	Infraestrutura	Mão de Obra	TOTAL
1-3	Desenvolvimento	¥0	R\$200/semana	R\$200/sem
4-6	Beta	¥250/mês	R\$200/semana	R\$200/sem
7-12	Produção	¥2,500/mês	R\$200/semana	R\$200/sem

*Na fase de produção (M7+), será necessário upgrade para Vercel Pro + Supabase Pro

3.3 Custos Iniciais (Setup)

Item	Custo (USD)	Uma vez?
Domínio (.jp)	~\$30	Sim (renovação anual)
Logo design (opcional)	\$0-500	Uma vez
Consultoria jurídica (opcional)	\$500-2000	Uma vez
Total inicial	\$530-2530	-

3.4 Custo de Mão de Obra (Engenheiro de TI)

Para manutenção e desenvolvimento contínuo do sistema, precisamos considerar o custo de mão de obra.

Cenário Atual: Desenvolvimento Próprio

Período	Horas/Semana	Valor Semanal	Valor Mensal
Desenvolvimento	10-20 hrs	R\$200	R\$800-1,600

Valor de R\$200/semana (~R\$32/hora para 6-12 horas/semana) para desenvolvimento e manutenção.

Custos Totais (Com Mão de Obra)

Fase	Infraestrutura	Mão de Obra	TOTAL/mês
Desenvolvimento	R\$0	R\$200 (~\$40)	R\$200 (~\$40)
Beta (M4-6)	R\$50	R\$200 (~\$40)	R\$250 (~\$50)
Produção (M7+)	R\$150-400	R\$200 (~\$40)	R\$350-600 (~\$70-120)

3.5 Estudo Comparativo: Supabase vs AWS Serverless

Fizemos uma análise para comparar a infraestrutura atual (Supabase + Vercel) com uma alternativa em AWS serverless. Detalhes completos estão no arquivo `AWS-COST-STUDY.md`.

Resumo da Comparação

Cenário	Supabase + Vercel	AWS Serverless	Melhor opção
Início (M1-3)	\$59/mês	\$21-55/mês	AWS
Crescimento (M6)	\$59/mês	\$71-119/mês	Supabase
Escala (M12)	\$80-100/mês	\$227-313/mês	Supabase

Por que Supabase é melhor para nós?

1. **Custo-benefício em escala inicial:** \$59 fixo vs \$21-55 variável
2. **Setup rápido:** Hours vs semanas no AWS
3. **Tudo integrado:** DB + Auth + Storage + Realtime em um serviço
4. **Japan region:** Latência baixa para usuários japoneses
5. **Zero manutenção:** Não preciso configurar Lambda, API Gateway, etc

Quando AWS compensaria?

- Volume muito alto (>500k requests/mês)
- Uso muito errático (bursts com períodos de ociosidade)
- Necessidade de serviços AWS específicos (SQS, DynamoDB, etc)

Componentes AWS necessários para substituir Supabase + Vercel:

Serviço AWS	Substitui	Custo base
Lambda	Edge Functions	\$0-10/mês
API Gateway	API Layer	\$1-10/mês
Aurora Serverless	PostgreSQL	\$16-80/mês
S3	Storage	\$1-5/mês
CloudFront	CDN	\$1-10/mês
Cognito	Auth	\$0-5/mês
CloudWatch	Logs	\$5-100/mês

Conclusão

Manter Supabase + Vercel é a melhor opção para o stage atual do projeto. AWS serverless só se beneficiaria em altíssima escala (>100k usuários ativos/mês).

5. Escalabilidade

5.1 O que acontece quando crescermos?

Recurso	Free (atual)	Pro (agora)	6 meses	12 meses
DB Storage	500 MB	8 GB	15 GB	50 GB
Bandwidth	2 GB	50 GB	150 GB	500 GB
Realtime Conn	50	200	500	2,000
Storage (imagens)	1 GB	100 GB	250 GB	500 GB
Edge Functions	500K/mês	2M/mês	5M/mês	10M/mês

5.2 Quando fazer upgrade?

- **Plano Pro → Enterprise:** Quando DB > 8GB ou bandwidth > 50GB/mês
- **Vercel Pro → Enterprise:** Quando precisa de mais analytics ou SSR

6. Plano de Desastre (DR)

6.1 O que é RTO/RPO?

- **RTO (Recovery Time Objective):** Quanto tempo para voltar a funcionar
- **RPO (Recovery Point Objective):** Quanto tempo de dados podemos perder

Serviço	RTO	RPO
Frontend	< 5 min (deploy automático)	N/A (código no git)
Database	< 1 hora	7 dias (backup automático)

Serviço	RTO	RPO
Edge Functions	< 10 min (redesploy)	N/A (código no git)

6.2 Cenários de Falha

Cenário	Probabilidade	Mitigação
Vercel fora do ar	Baixa	DNS backup (Netlify)
Banco de dados corrompido	Baixa	Restore do backup
Ataque DDoS	Média	Cloudflare protection
Stripe falha	Baixa	Retry logic + logs

7. Checklist de Produção

Pré-Deploy (antes de lançar)

- [] Domínio registrado e apontando para Vercel
- [] SSL funcionando (<https://japancarparts.jp>)
- [] Todas variáveis de ambiente configuradas
- [] Build passa localmente (`npm run build`)
- [] Lint e typecheck passando

Deploy

- [] Frontend deployed na Vercel
- [] Edge Functions deployed no Supabase
- [] Banco de dados schema aplicado
- [] Políticas RLS aplicadas
- [] Webhook Stripe configurado

Pós-Deploy (testar)

- [] Homepage carregando
- [] Login/registro funcionando
- [] Criar peça funcionando

- [] Upload de imagens funcionando
- [] Monitoramento configurado

Resumo Executivo

Item	Valor
Custo mensal inicial	~\$59 USD (¥8,750)
Tech stack	React + Vite + Supabase + Stripe
Domínio	japancarparts.jp
RTO (recuperação)	< 5 min (frontend), < 1h (db)
Escalabilidade	Suporta até 50k usuários no plano atual

Documento elaborado em: 2026-05-09 Projeto: JAPANCAR PARTS Marketplace Versão: 1.0.0